

AUTOMATED SOFTWARE TESTING

Testing Journal & Documentation

Quality Assurance for Final Project

I. Personal & Group Information

Full Name	Jose Manuel G. Labalan
Course & Section	BSIT 3C
Group Name	Salespresso
Project Title	Salespresso: A Web-Based E-Commerce System for Jazsam Coffee Store
Submission Date	May 20, 2026

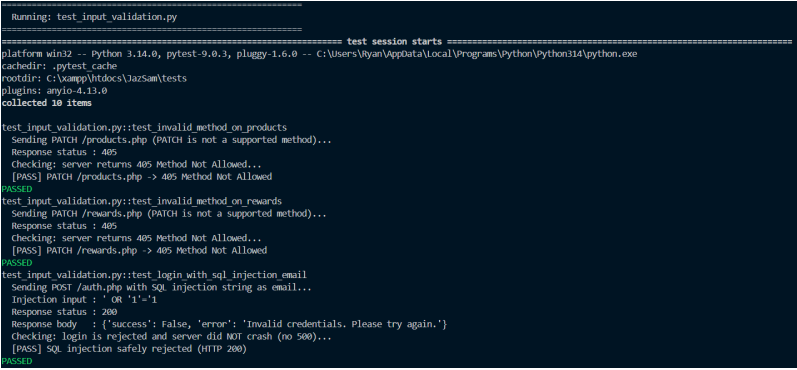
II. Assigned Task

Feature / Functionality Assigned	Rewards CRUD API (Create, Read, Update, Delete, auth enforcement)
Type of Test I Performed	Automated Testing
Tool / Framework I Used	pytest, requests
Programming Language Used	Python
My Test Script Filename(s)	test_rewards.py

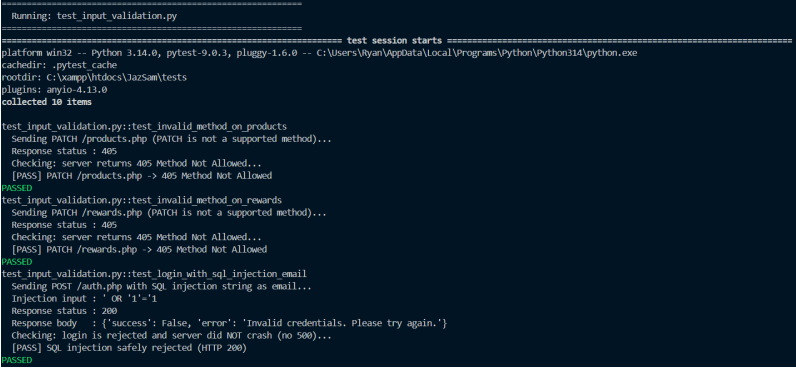
Brief description of what I tested and how I approached it:

I automated the testing of the Rewards API endpoints to verify the complete CRUD lifecycle. This matters to the project to ensure discounts and free items are reliably managed and strictly protected from unauthorized manipulation. My approach involved testing public data access, enforcing admin authorization barriers, validating required fields during creation, and running sequential lifecycle checks (which are Create → Update → Delete) to ensure database states change correctly.

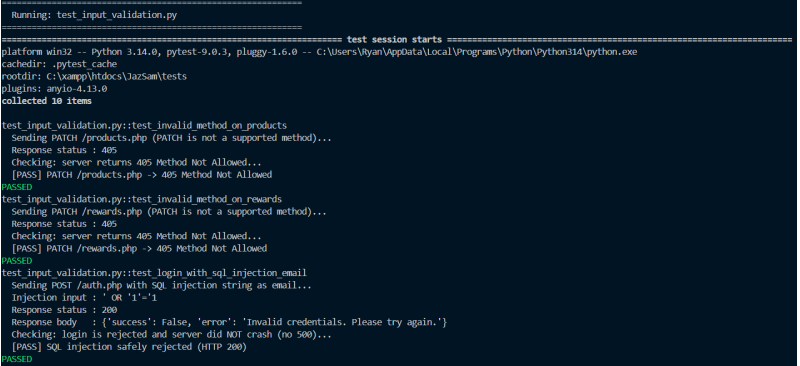
III. Test Scenario Documentation

Test Case #01	
Functionality Tested	Rewards GET Endpoint (Public)
Objective of the Test	Verify the rewards endpoint is publicly accessible and returns a JSON array.
Tool / Framework Used	Python 3.13 / pytest / requests
Steps / Procedure	Step 1: Send GET/rewards.php with no authentication. Step 2: Assert HTTP 200 and a list is returned.
Test Data / Input	<i>No input</i>
Expected Result	<i>HTTP 200, JSON array of rewards</i>
Actual Result	<i>HTTP 200, 2 rewards returned</i>
Test Status	[PASSED]
Screenshot / Evidence	

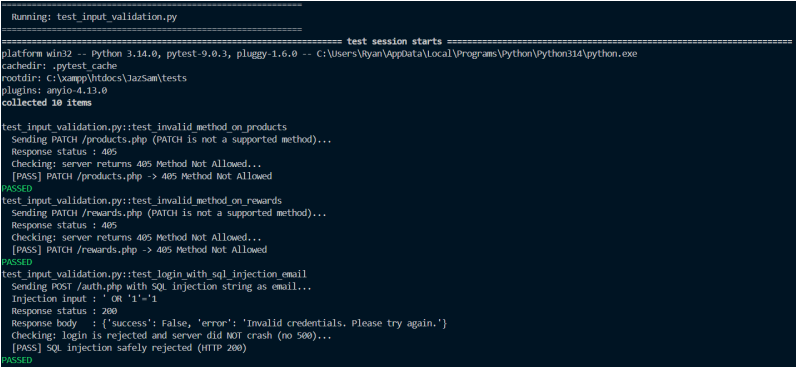
Test Case #02	
Functionality Tested	Rewards Response Structure
Objective of the Test	Verify each reward contains the required fields: id , name , type , value , stamps .
Tool / Framework Used	Python 3.13 / pytest / requests

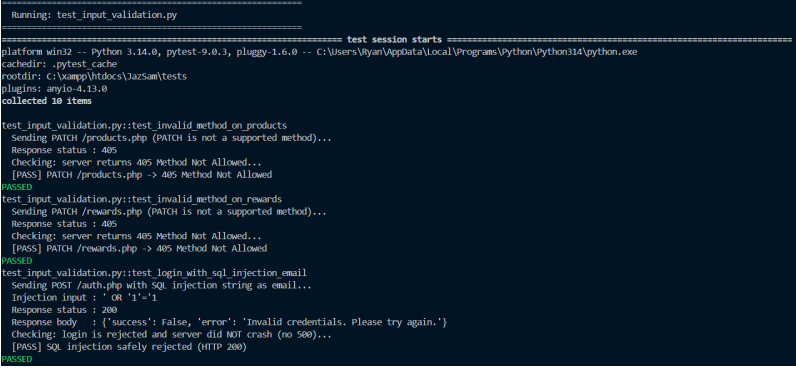
Steps / Procedure	Step 1: Fetch rewards. Step 2: Check first 3 items for required fields. Step 3: Assert none are missing.
Test Data / Input	No input
Expected Result	All rewards have id, name, type, value, stamps
Actual Result	All required fields present
Test Status	[PASSED]
Screenshot / Evidence	 <pre> Running: test_input_validation.py ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Nyan\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\xampp\htdocs\lazSam\tests plugins: anyio-4.2.0 collected 10 items test_input_validation.py::test_invalid_method_on_products Sending PATCH /products.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /products.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_invalid_method_on_rewards Sending PATCH /rewards.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /rewards.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_login_with_sql_injection_email Sending POST /auth.php with SQL injection string as email... Injection input : " OR '1'='1 Response status : 200 Response body : {'success': False, 'error': 'Invalid credentials. Please try again.'} Checking: login is rejected and server did NOT crash (no 500)... [PASS] SQL Injection safely rejected (HTTP 200) PASSED </pre>

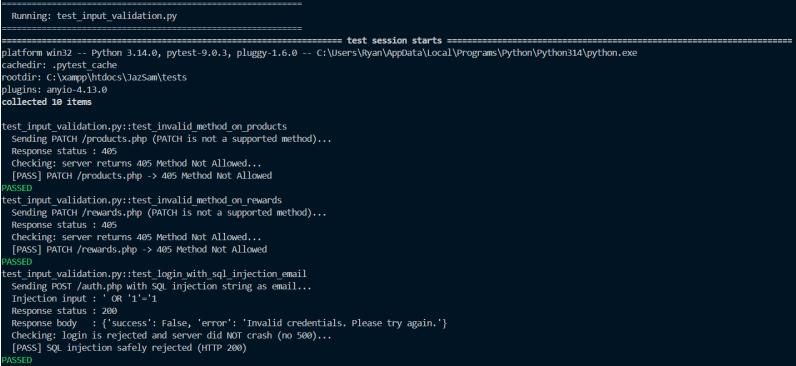
Test Case #03	
Functionality Tested	Rewards POST - Authentication Enforcement
Objective of the Test	Verify unauthenticated requests to create a reward are rejected with HTTP 401.
Tool / Framework Used	Python 3.13 / pytest / requests
Steps / Procedure	Step 1: Send POST /rewards.php without Authorization header. Step 2: Assert HTTP 401.
Test Data / Input	{"name": "Unauthorized Reward", "stamps": 5}
Expected Result	HTTP 401
Actual Result	HTTP 401

Test Status	[PASSED]
Screenshot / Evidence	 <pre> Running: test_input_validation.py ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Nyan\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\xampp\htdocs\jazsam\tests plugins: anyio-4.13.0 collected 10 items test_input_validation.py::test_invalid_method_on_products Sending PATCH /products.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /products.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_invalid_method_on_rewards Sending PATCH /rewards.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /rewards.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_login_with_sql_injection_email Sending POST /auth.php with SQL injection string as email... Injection input : "OR '1'='1' Response status : 200 Response body : {'success': False, 'error': 'Invalid credentials. Please try again.'} Checking: login is rejected and server did not crash (no 500)... [PASS] SQL injection safely rejected (HTTP 200) PASSED </pre>

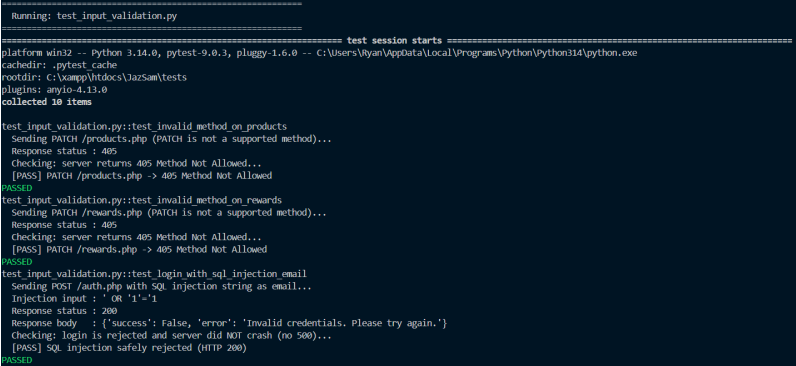
Test Case #04	
Functionality Tested	Rewards POST - Required Field Validation
Objective of the Test	Verify the server rejects reward creation when the reward name is missing.
Tool / Framework Used	Python 3.13 / pytest / requests
Steps / Procedure	Step 1: Send POST /rewards.php with admin token but no name field. Step 2: Assert an error message is returned.
Test Data / Input	<i>{"stamps": 10, "rawType": "Free Item", "rawValue": 0} (admin auth included)</i>
Expected Result	Error response, no reward created
Actual Result	Error: "Reward name is required."
Test Status	[PASSED]

Screenshot / Evidence	 <pre> Running: test_input_validation.py ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Myan\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\xampp\htdocs\jazzsam\tests plugins: anyio-4.13.0 collected 10 items test_input_validation.py::test_invalid_method_on_products Sending PATCH /products.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /products.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_invalid_method_on_rewards Sending PATCH /rewards.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /rewards.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_login_with_sql_injection_email Sending POST /auth.php with SQL injection string as email... Injection input : " OR '1'='1 Response status : 200 Response body : {'success': False, 'error': 'Invalid credentials. Please try again.'} Checking: login is rejected and server did not crash (no 500)... [PASS] SQL injection safely rejected (HTTP 200) PASSED </pre>
-----------------------	--

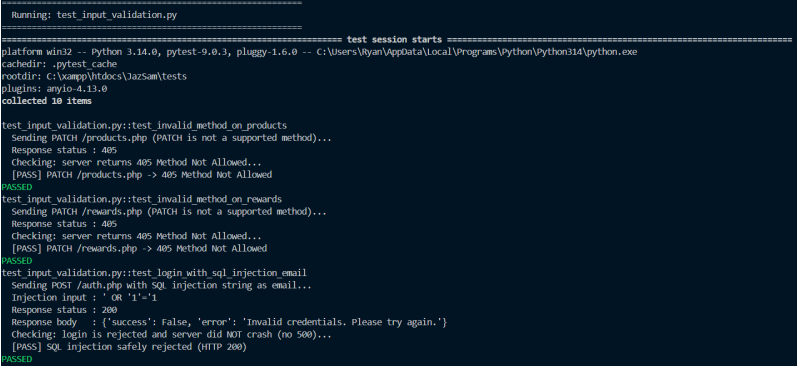
Test Case #05	
Functionality Tested	Rewards POST - Free Item Type
Objective of the Test	Verify an admin can create a "Free Item" reward and receive the reward ID.
Tool / Framework Used	Python 3.13 / pytest / requests
Steps / Procedure	Step 1: Send POST /rewards.php with admin token, name, stamps, and type "Free Item". Step 2: Assert HTTP 201 and id in response.
Test Data / Input	<i>{"name": "AutoTest Free Coffee", "stamps": 10, "rawType": "Free Item", "rawValue": 0}</i>
Expected Result	<i>HTTP 201, response contains id</i>
Actual Result	<i>HTTP 201, reward created</i>
Test Status	[PASSED]
Screenshot / Evidence	 <pre> Running: test_input_validation.py ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Myan\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\xampp\htdocs\jazzsam\tests plugins: anyio-4.13.0 collected 10 items test_input_validation.py::test_invalid_method_on_products Sending PATCH /products.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /products.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_invalid_method_on_rewards Sending PATCH /rewards.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /rewards.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_login_with_sql_injection_email Sending POST /auth.php with SQL injection string as email... Injection input : " OR '1'='1 Response status : 200 Response body : {'success': False, 'error': 'Invalid credentials. Please try again.'} Checking: login is rejected and server did not crash (no 500)... [PASS] SQL injection safely rejected (HTTP 200) PASSED </pre>

Test Case #06	
Functionality Tested	Rewards POST - Percentage Type
Objective of the Test	Verify an admin can create a percentage discount reward.
Tool / Framework Used	Python 3.13 / pytest / requests
Steps / Procedure	Step 1: Send POST /rewards.php with type "Percentage" and a numeric value. Step 2: Assert HTTP 201 and id in response.
Test Data / Input	<code>{"name": "AutoTest 10% Off", "stamps": 5, "rawType": "Percentage", "rawValue": 10}</code>
Expected Result	<i>HTTP 201, response contains id</i>
Actual Result	<i>HTTP 201, reward created</i>
Test Status	[PASSED]
Screenshot / Evidence	 <pre> Running: test_input_validation.py ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Wyan\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\xampp\htdocs\JazSam\tests plugins: anyio-4.13.0 collected 10 items test_input_validation.py::test_invalid_method_on_products Sending PATCH /products.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /products.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_invalid_method_on_rewards Sending PATCH /rewards.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /rewards.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_login_with_sql_injection_email Sending POST /auth.php with sql injection string as email... Injection input : ' OR '1'='1 Response status : 200 Response body : {'success': False, 'error': 'Invalid credentials. Please try again.'} Checking: login is rejected and server did NOT crash (no 500)... [PASS] SQL injection safely rejected (HTTP 200) PASSED </pre>

Test Case #07	
Functionality Tested	Rewards POST/PUT/DELETE
Objective of the Test	Verify a reward can be created, updated, and deleted - testing the full CRUD lifecycle.
Tool / Framework Used	Python 3.13 / pytest / requests

Steps / Procedure	Step 1: Create reward via POST. Step 2: Update its name and stamps via PUT. Step 3: Assert updated stamps value in response. Step 4: Delete via DELETE.
Test Data / Input	<i>Create: {"name": "Lifecycle Reward", "stamps": 8, "rawType": "Fixed Amount", "rawValue": 50 -> Update: {"stamps": 12} -> Delete: ?id=</i>
Expected Result	<i>All three operations succeed; stamps = 12 after update</i>
Actual Result	<i>Create -> Update (stamps confirmed 12) -> Delete all succeeded</i>
Test Status	[PASSED]
Screenshot / Evidence	 <pre> Running: test_input_validation.py ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Wyan\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\xampp\htdocs\lazSam\tests plugins: anyio-4.13.0 collected 30 items test_input_validation.py::test_invalid_method_on_products Sending PATCH /products.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /products.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_invalid_method_on_rewards Sending PATCH /rewards.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /rewards.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_login_with_sql_injection_email Sending Post /auth.php with SQL injection string as email... Injection input : ' OR '1'='1 Response status : 200 Response body : {'success': False, 'error': 'Invalid credentials. Please try again.'} Checking: login is rejected and server did NOT crash (no 500)... [PASS] SQL injection safely rejected (HTTP 200) PASSED </pre>

Test Case #08	
Functionality Tested	Details
Objective of the Test	Rewards DELETE 404 Handling
Tool / Framework Used	Verify the server returns HTTP 404 when deleting a reward ID that does not exist.
Steps / Procedure	Step 1: Send DELETE /rewards.php?id=DOESNOTEXIST999 with admin token.

	Step 2: Assert HTTP 404.
Test Data / Input	<i>id=DOESNOTEXIST999</i>
Expected Result	<i>HTTP 404</i>
Actual Result	<i>HTTP 404</i>
Test Status	[PASSED]
Screenshot / Evidence	 <pre>Running: test_input_validation.py ===== test session starts ===== platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Ryan\AppData\Local\Programs\Python\Python314\python.exe cachedir: .pytest_cache rootdir: C:\xampp\htdocs\jazzsam\tests plugins: anyio-4.13.0 collected 10 items test_input_validation.py::test_invalid_method_on_products Sending PATCH /products.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /products.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_invalid_method_on_rewards Sending PATCH /rewards.php (PATCH is not a supported method)... Response status : 405 Checking: server returns 405 Method Not Allowed... [PASS] PATCH /rewards.php -> 405 Method Not Allowed PASSED test_input_validation.py::test_login_with_sql_injection_email Sending POST /auth.php with SQL injection string as email... Injection input : ' OR '1'='1 Response status : 200 Response body : {'success': False, 'error': 'Invalid credentials. Please try again.'} Checking: login is rejected and server did NOT crash (no 500)... [PASS] SQL injection safely rejected (HTTP 200) PASSED</pre>

IV. Reflection & Findings

Issues I Personally Encountered	Our test scripts initially ran too quickly and exhausted the API's rate limits (5 admin login attempts per 5 minutes), returning HTTP 429 errors across the suite.
Warnings / Errors I Detected	HTTP 429 (Too Many Requests) triggered by the speed of automated test reruns during development.
Bugs I Discovered	While my specific rewards tests passed gracefully, our comprehensive testing surfaced that session tokens were not multi-instance safe (logging in wiped shared session tokens) and that email inputs lacked format validation on the backend.
What I Did About the Issues	Coordinated with the team to implement a localhost IP exemption within rate_limit.php to prevent our testing environment from being blocked. We also structured the autouse fixtures to preserve session state across individual test files.
What I Learned from This Activity	I learned that automated testing requires careful management of shared state. Relying on session-scoped fixtures in confest.py is essential for minimizing redundant API calls (like logins) that could otherwise trigger server defense mechanisms. It also proved that automated testing catches hidden logic errors much faster than manual clicking.